Assignment 2: Parallel Efficiency — MPI Monte Carlo π

Jinye Gong
jinyeg@kth.se

Weiyi Lyu
weiyil@kth.se

2026-04-18

Contributions

The work in this assignment was shared equally between the two group members. We divided the experiments on Dardel, the school cluster, analysis, plotting, and report writing evenly, and both contributed throughout the whole process.

Exercise 1: Calculate Parallel Efficiency

Task 1: Dardel (compute nodes, batch jobs)

1. Steps for compiling the code on Dardel

We uploaded `pi_mpi.c` and `Makefile` to `~/dd2356_a2` on Dardel. On the login node, the default Cray programming environment already provides MPI compiler wrappers. We built two binaries from the same source: one for strong scaling (default) and one for weak scaling (macro `ASSIGNMENT_WEAK_SCALING`).

```
cd ~/dd2356_a2
make clean
make CC=cc pi_mpi_strong pi_mpi_weak
```

This produces `pi_mpi_strong` and `pi_mpi_weak`. We use `cc` because it is the Cray MPI wrapper on Dardel and links against the correct `cray-mpich` libraries.

2. Slurm commands used for executing the tests

We submitted batch jobs with our course allocation. The project account was taken from `projinfo` (`edu26.dd2356`). Each script requests up to 16 MPI tasks on the `shared` partition, loops over $n \in {1,2,4,8,16}$, and launches the program with `srun`.

```
chmod +x slurm_strong.sh slurm_weak.sh
sbatch slurm_strong.sh
sbatch slurm_weak.sh
```

The job scripts contain directives such as:

```
#SBATCH -J pi_strong
#SBATCH -A edu26.dd2356
#SBATCH -p shared
#SBATCH -n 16
#SBATCH -t 0:15:00
#SBATCH -o strong_%j.out
```

Inside the script, strong scaling runs:

```
srun -n "${n}" ./pi_mpi_strong 10000000
```

and weak scaling runs:

```
srun -n "${n}" ./pi_mpi_weak
```

We inspected results with `cat strong_<JOBID>.out` and `cat weak_<JOBID>.out`.

3. Strong scaling — code in the region for Option 1 (total tosses = 10 million)

The total number of tosses is fixed. Each rank receives a fair share; ranks `0 ... (tosses % nproc - 1)` get one extra toss when the total is not divisible. In our experiments, 10^7 is divisible by all tested `nproc`, but the code is written in a general way.

```
local_tosses = tosses / nproc + (rank < (tosses % nproc) ? 1 : 0);
total_tosses = tosses;
```

4. Strong scaling — screenshot of the output (Dardel)

Figure 1 shows the Slurm output for strong scaling on Dardel for 1, 2, 4, 8, and 16 processes with a total of 10,000,000 tosses (batch job `19763955`, file `strong_19763955.out`).

```
==============================
Strong scaling: processes = 1, total tosses = 10000000
==============================
Estimated Pi: 3.141855600000, True Pi: 3.141592653590
Error: 0.000262946410
Total Tosses: 10000000
Execution Time: 0.175691 seconds

==============================
Strong scaling: processes = 2, total tosses = 10000000
==============================
Estimated Pi: 3.141719200000, True Pi: 3.141592653590
Error: 0.000126546410
Total Tosses: 10000000
Execution Time: 0.085037 seconds

==============================
Strong scaling: processes = 4, total tosses = 10000000
==============================
Estimated Pi: 3.141487200000, True Pi: 3.141592653590
Error: 0.000105453590
Total Tosses: 10000000
Execution Time: 0.043196 seconds

==============================
Strong scaling: processes = 8, total tosses = 10000000
==============================
Estimated Pi: 3.141280000000, True Pi: 3.141592653590
Error: 0.000312653590
Total Tosses: 10000000
Execution Time: 0.022678 seconds

==============================
Strong scaling: processes = 16, total tosses = 10000000
==============================
Estimated Pi: 3.141384400000, True Pi: 3.141592653590
Error: 0.000208253590
Total Tosses: 10000000
Execution Time: 0.012897 seconds
```

5. Weak scaling — code in the region for Option 2 (local tosses = 1 million per process)

Each rank performs exactly 1,000,000 local tosses. The global total is $nproc \times 1{,}000{,}000$, which is used as the denominator when rank 0 estimates π.

```
local_tosses = 1000000LL;
total_tosses = (long long int)nproc * local_tosses;
(void)tosses; /* argv[1] total not used in weak scaling */
```

6. Weak scaling — screenshot of the output (Dardel)

Figure 2 shows the Slurm output for weak scaling on Dardel for 1, 2, 4, 8, and 16 processes (batch job `19763960`, file `weak_19763960.out`).

```
============================
Weak scaling: processes = 1, local tosses = 1000000 per rank
============================
srun: warning: can't run 1 processes on 2 nodes, setting nnodes to 1
Estimated Pi: 3.141736000000, True Pi: 3.141592653590
Error: 0.000143346410
Total Tosses: 1000000
Execution Time: 0.017239 seconds

============================
Weak scaling: processes = 2, local tosses = 1000000 per rank
============================
Estimated Pi: 3.140516000000, True Pi: 3.141592653590
Error: 0.001076653590
Total Tosses: 2000000
Execution Time: 0.019204 seconds

============================
Weak scaling: processes = 4, local tosses = 1000000 per rank
============================
Estimated Pi: 3.140587000000, True Pi: 3.141592653590
Error: 0.001005653590
Total Tosses: 4000000
Execution Time: 0.017522 seconds

============================
Weak scaling: processes = 8, local tosses = 1000000 per rank
============================
Estimated Pi: 3.141875500000, True Pi: 3.141592653590
Error: 0.000282846410
Total Tosses: 8000000
Execution Time: 0.018024 seconds

============================
Weak scaling: processes = 16, local tosses = 1000000 per rank
============================
Estimated Pi: 3.142514000000, True Pi: 3.141592653590
Error: 0.000921346410
Total Tosses: 16000000
Execution Time: 0.025563 seconds
```

Task 2: School cluster ("DD2356 — Medium CPU only")

We started a Jupyter session with the **DD2356 - Medium CPU only** server option, opened a terminal as user `jovyan`, and used the same `pi_mpi.c` and `Makefile` under `~/dd2356_a2`.

1. Compile and run

```
cd ~/dd2356_a2
make CC=mpicc pi_mpi_strong pi_mpi_weak
```

Strong scaling:

```
for n in 1 2 4 8 16; do
  echo "---- processes = $n ----"
  mpirun -np $n ./pi_mpi_strong 10000000
done
```

Weak scaling:

```
for n in 1 2 4 8 16; do
  echo "---- processes = $n ----"
```

```
  mpirun -np $n ./pi_mpi_weak
done
```

2. Screenshots

Figure 3: Strong scaling output on the school cluster (Jupyter).

```
======== STRONG ========
---- processes = 1 ----
Estimated Pi: 3.142062800000, True Pi: 3.141592653590
Error: 0.000470146410
Total Tosses: 10000000
Execution Time: 0.275108 seconds
---- processes = 2 ----
Estimated Pi: 3.141542400000, True Pi: 3.141592653590
Error: 0.000050253590
Total Tosses: 10000000
Execution Time: 0.137692 seconds
---- processes = 4 ----
Estimated Pi: 3.140879200000, True Pi: 3.141592653590
Error: 0.000713453590
Total Tosses: 10000000
Execution Time: 0.074518 seconds
---- processes = 8 ----
Estimated Pi: 3.141275600000, True Pi: 3.141592653590
Error: 0.000317053590
Total Tosses: 10000000
Execution Time: 0.059016 seconds
---- processes = 16 ----
Estimated Pi: 3.142012000000, True Pi: 3.141592653590
Error: 0.000419346410
Total Tosses: 10000000
Execution Time: 0.028300 seconds
jovyan@jupyter-jinyeg:~/dd2356_a2$
```

Figure 4: Weak scaling output on the school cluster (Jupyter).

```
======== WEAK ========
---- processes = 1 ----
Estimated Pi: 3.140540000000, True Pi: 3.141592653590
Error: 0.001052653590
Total Tosses: 1000000
Execution Time: 0.027189 seconds
---- processes = 2 ----
Estimated Pi: 3.139962000000, True Pi: 3.141592653590
Error: 0.001630653590
Total Tosses: 2000000
Execution Time: 0.027208 seconds
---- processes = 4 ----
Estimated Pi: 3.141783000000, True Pi: 3.141592653590
Error: 0.000190346410
Total Tosses: 4000000
Execution Time: 0.032771 seconds
---- processes = 8 ----
Estimated Pi: 3.142984500000, True Pi: 3.141592653590
Error: 0.001391846410
Total Tosses: 8000000
Execution Time: 0.048876 seconds
---- processes = 16 ----
Estimated Pi: 3.142156750000, True Pi: 3.141592653590
Error: 0.000564096410
Total Tosses: 16000000
Execution Time: 0.048557 seconds
jovyan@jupyter-jinyeg:~/dd2356_a2$
```

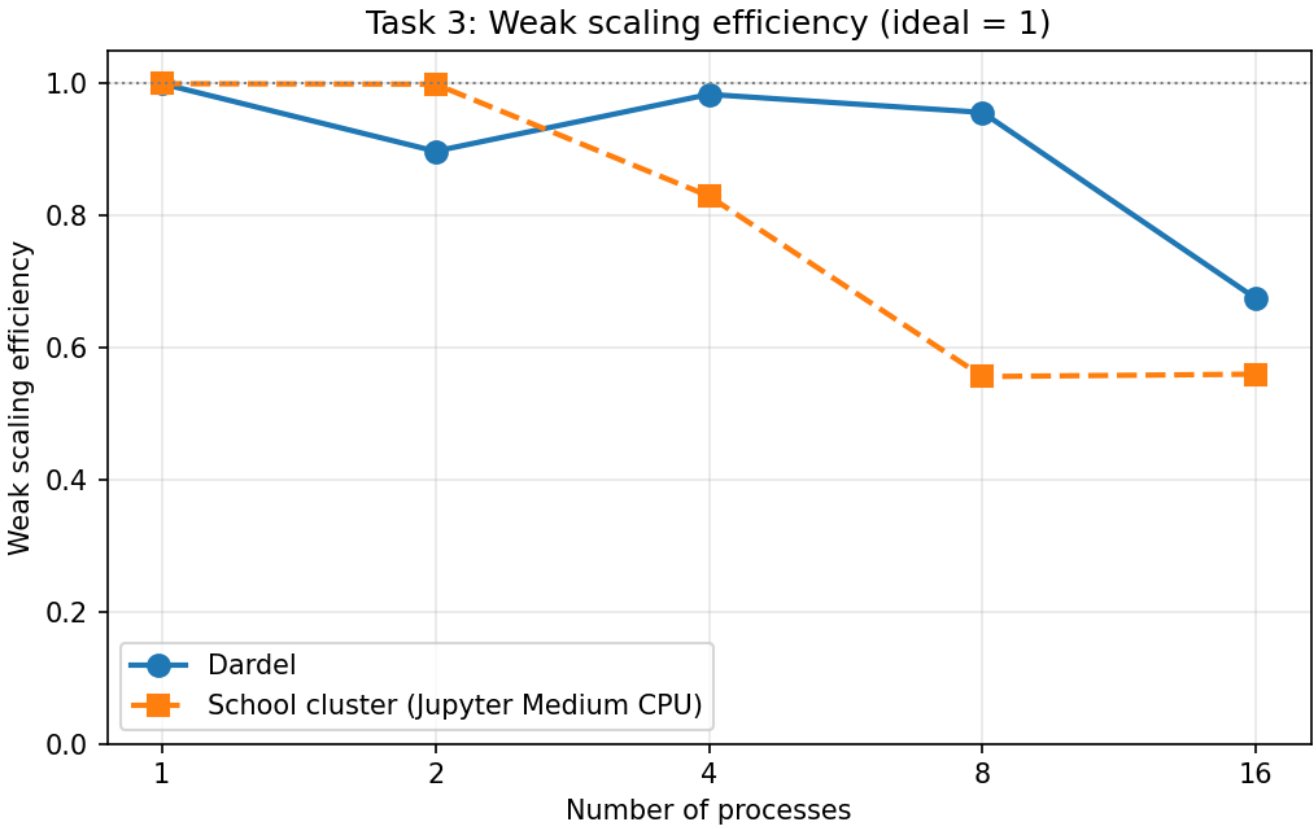
Task 3: Weak scaling efficiency (both systems, one chart)

Weak scaling uses the following measure:

$$ E_{\mathrm{weak}}(p) = \frac{T_1}{T_p} $$

Here, T_1 is the wall-clock time with one MPI process using the baseline local workload, and T_p is the wall-clock time with p MPI processes when each process keeps the same local workload. In this assignment, that workload is 1,000,000 tosses per process. Ideal weak scaling means that the runtime stays constant as p grows, so $E_{\mathrm{weak}}(p) = 1$.

Figure 5 plots $E_{\mathrm{weak}}(p)$ for Dardel and the school Jupyter environment for $p \in {1,2,4,8,16}$.



Observations (1–2 sentences):
Both systems stay close to ideal scaling at small p, but the curve falls at larger p, especially on the school Jupyter node, because of MPI overhead and contention on a shared medium-CPU node. Dardel keeps a higher weak-scaling efficiency at 16 processes.

Numerical values:

p	Dardel	Jupyter
1	1.0000	1.0000
2	0.8977	0.9993
4	0.9838	0.8297
8	0.9564	0.5563
16	0.6744	0.5599

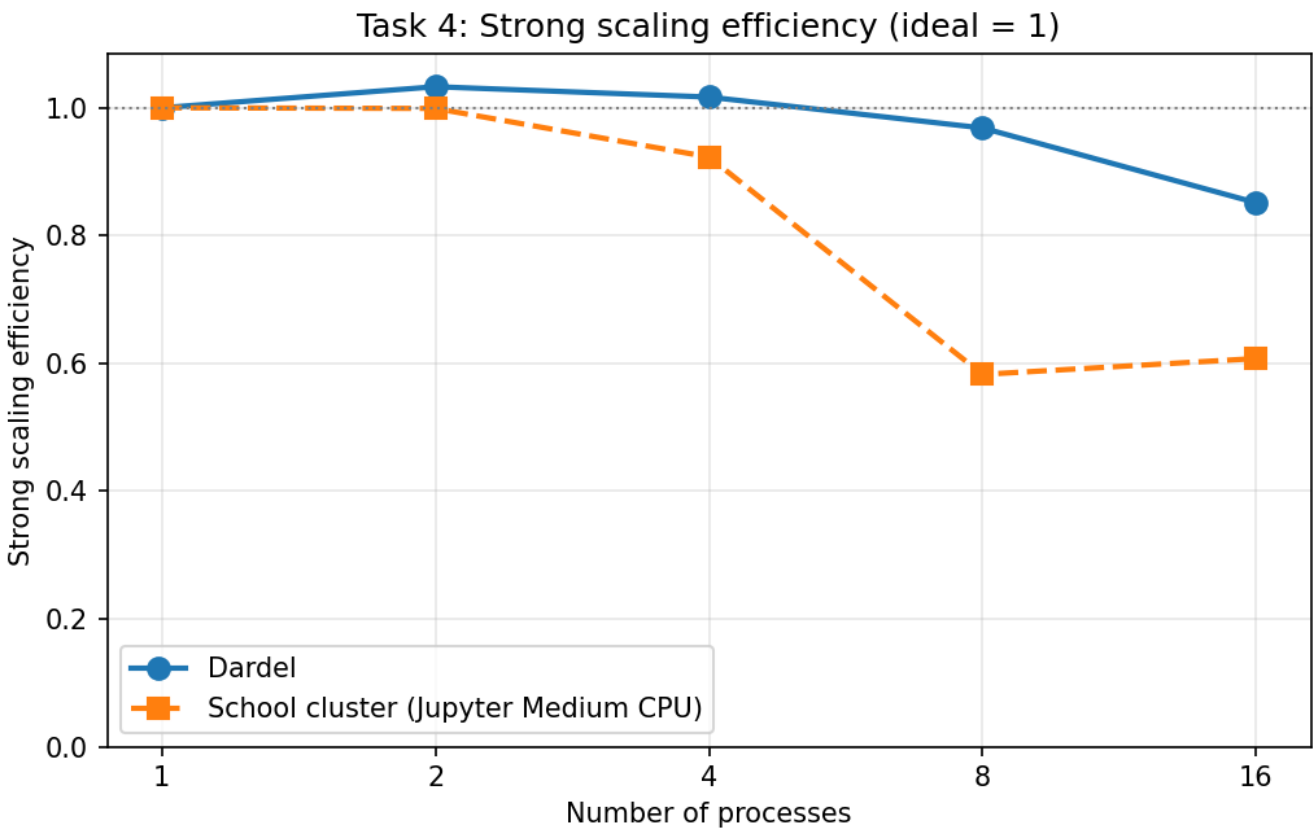
Task 4: Strong scaling efficiency (both systems, one chart)

Strong scaling uses the following measure:

$$ E_{\mathrm{strong}}(p) = \frac{T_1}{p , T_p} $$

Here, T_1 is the wall-clock time with one MPI process, T_p is the wall-clock time with p processes, and the global problem size stays the same when p grows. In this assignment, the global problem size is 10,000,000 tosses in total. The product p , T_p represents the total core-time if all ranks run for time T_p. Ideal strong scaling means $E_{\mathrm{strong}}(p) = 1$.

Figure 6 plots $E_{\mathrm{strong}}(p)$ for both systems.



Observations (1–2 sentences):
On Dardel, $E_{\mathrm{strong}}(p)$ stays near or slightly above 1 for small p, which is likely due to timing noise, and then drops to about 0.85 at 16 ranks. On the school cluster, it falls earlier, especially around 8 to 16 ranks, which is consistent with MPI overhead and resource sharing on one interactive node.

Numerical values:

p	Dardel	Jupyter
1	1.0000	1.0000
2	1.0330	0.9990
4	1.0168	0.9230
8	0.9684	0.5827
16	0.8514	0.6076